

4/13/2026

Enterprise Application Development Part 1

Surname & Name: Murendi Tendani Mary

Student Number:ST10401447

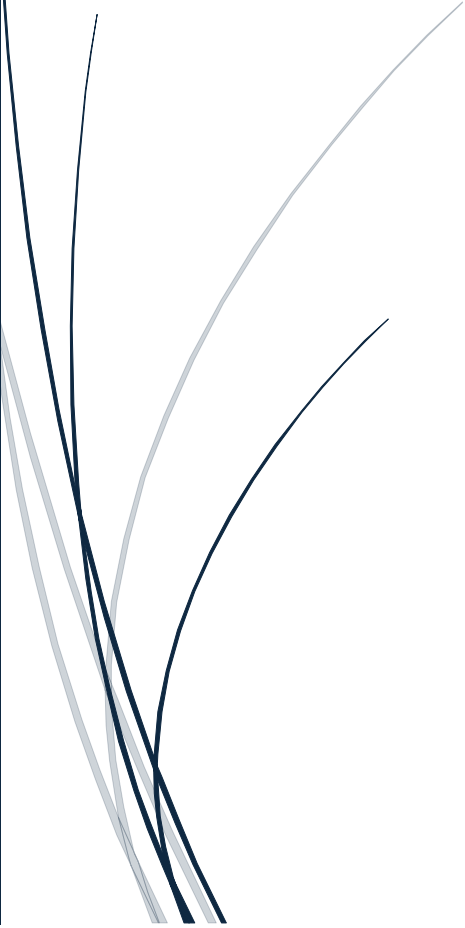


Table of Contents

1.Executive Summary &Introduction	2
Why Zachman Framework was chosen	2
Zachman Framework: GLMS Planner and Designer View	2
2.Design Pattern Selection	2
Pattern 1:Factory Method(Creational).....	3
Pattern 2:Observer(Behavioural)	4
Pattern 3:Strategy(Behavioural)	4
3.Non-Functional Requirements & Scalability	4
3.1. Availability	4
3.2.Interoperability	5
3.3.Scalability strategy	5
Strategy 1:Horizontal scaling.....	5
Strategy 2:Load Balancing.....	5
Strategy 3:Caching	5
Reference List:	7

1.Executive Summary &Introduction

TechMove Logistics currently relies on disconnected spreadsheets,manual emails and phone calls to manage international freight contracts,driver’s schedules and invoices. This has caused data fragmentation,lost invoices and compliance failures.This report addressed to the TechMove Board of directors presents a complete architectural blueprint for the Global Logistics Management System(GLMS) which is a modern web application that runs in the cloud and can grow as the business grows.The report covers three areas which are Zachman Framework applied across Planner and Designer perspectives,three GoF design patterns with UML diagrams and scalability strategy using horizontal scaling,load balancing and Redis caching to handle a sudden increase of 50 000 new contracts overnight.

Why Zachman Framework was chosen

The Zachman Framework was chosen to organise the GLMS architecture.It is a grid where rows represent different viewpoints and columns answer the key questions (Edmead,2025).The planner row captures the business view where the system stores contracts,clients,invoices,service requests,manages freight operations and currency conversion and connects TechMove offices to external APIs.The designer rows defines the technical detail which includes entities like Contract and ServiceRequest with specific fields,business rules such as,only active contracts can raise service request,status transitions from Draft to Active to Expired and a technical stack of ASP.NET Core MVC,SQL Server,REST APIs and cloud hosting.

Zachman Framework: GLMS Planner and Designer View

Perspective	Data(What)	Function(How)	Network(Where)
Planner(Scope)	Contracts, Clients, Invoices, Drivers, Service Request,Currency Rates	Freight Operation Management process	TechMove Head Office,Online Web Portal, International shipping location
Designer (Architect)	Logical Data Tables(Contract,ServiceRequest,Invoice, Client)	Business rules and Workflows	Web-based Distributed architecture

Figure 1:*Planner & Designer rows displaying Data,Function and network columns for the GLMS at TechMove Logistics*

2.Design Pattern Selection

According to Subbareddy(2025) design patterns are proven,reusable solutions to common problems in software design.They are not complete programs,but they are blueprints that guide how to structure a code.Using patterns Makes the system easier to maintain and test.Three GoF(Gang of four) patterns have been selected for the GLMS.

	Pattern	Category	GLMS Use Case
1.	Factory Method	Creational	Creating different types of contracts and service requests.
2.	Observer Method	Behavioural	Alerting managers whenever the status of the contract changes.
3.	Strategy Method	Behavioural	Applying different rules for currency conversion.

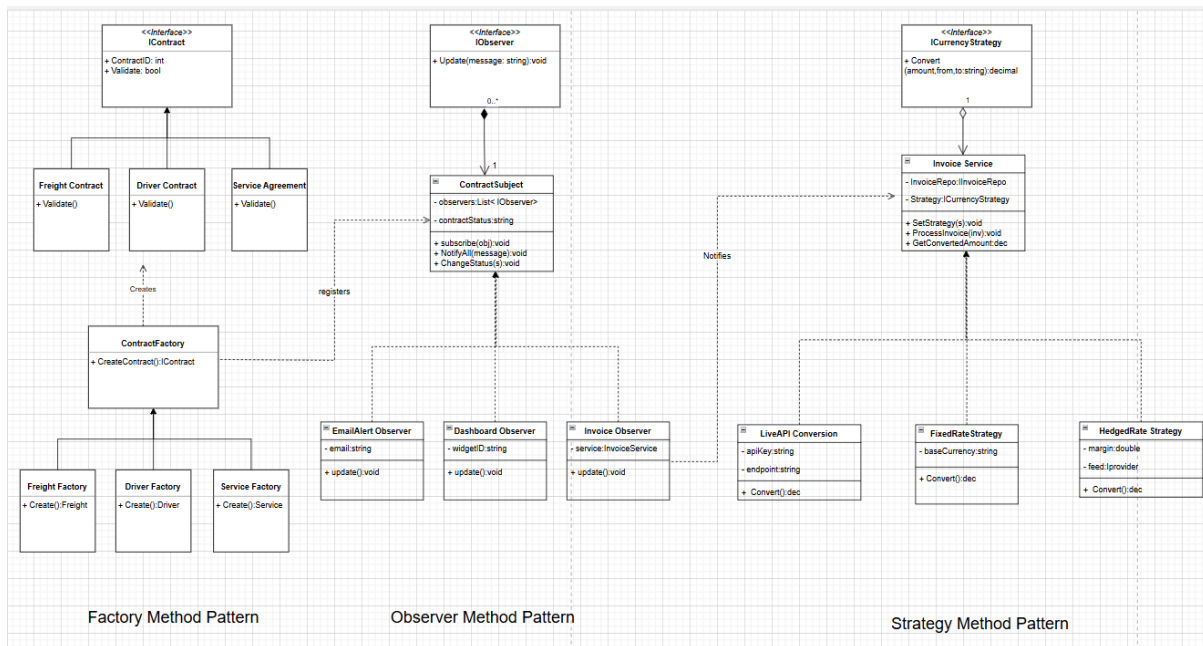


Figure 2: Design Patterns in GLMS-Factory Method, Observer and Strategy

Pattern 1: Factory Method (Creational)

Problem it solves: The GLMS must create different contract types which are `FreightContract`, `DriverContract` and `Service Agreement`. Hard coding which class to create makes the system fragile and adding a new contract type would require changing the existing controller code (geeksforgeeks, 2026).

How the diagram works: `Contract` is the interface that all contract types must implement, it guarantees every contract has a `ContractID` and a `Validate()` method. The three concrete contracts inherit it. `ContractFactory` is the abstract creator, its `CreateContract()` method returns `Contract`. The <<creates>> dashed arrows show this dependency. The three concrete factories inherit from `ContractFactory` and each override `CreateContract()` to produce their own specific type.

Main Benefit: The controller calls `factory.CreateContract()` without knowing which contract class it gets. New types only need a new factory subclass, no existing code changes.

Pattern 2:Observer(Behavioural)

Problem it solves: When a contract's status changes for example from Active to expired, multiple parts of the system need to react, send an email, update dashboard, write a compliance log and trigger the invoice processing. Without this pattern ContractSubject would have to call every system directly, creating coupling.

How the diagram works: IObserver declares one method which is update(message). Any class that wants to be modified must implement this method. ContractSubject holds a List<IObserver> shown by aggregation pointing to IObserver. The 0..* means that zero or more observers can register. When ChangeStatus() is called, it calls NotifyAll() which loops through every observer and calls update().

The cross-pattern connection: InvoiceObserver is a concrete observer that also holds a reference to InvoiceService. When notified, it calls ProcessInvoice() on InvoiceService, bridging the observer and strategy patterns. This means a contract status change automatically triggers currency converted invoice processing.

Pattern 3:Strategy(Behavioural)

Problem it solves: International invoices require different currency conversion methods depending on the contract type, a live API rate, a fixed corporate rate or a hedged rate. If the conversion logic is hard coded inside InvoiceService using if/else, adding a new method means modifying core business logic which is risky.

How the diagram works: ICurrencyStrategy declares Convert(amount, from, to). The three concrete strategies which are LiveAPIConversionStrategy, FixedRateStrategy, HedgedRateStrategy implement it. InvoiceService holds one strategy reference shown by aggregation. SetStrategy() allows the system swap the algorithm at runtime. When ProcessInvoice() is called, it calls strategy.Convert() without knowing which strategy is active.

The cross-pattern connection: InvoiceService is triggered by InvoiceObserver from the observer pattern. This means a full chain is the following: the contract status changes, the ContractSubject then notifies the observers, the Invoice observer then calls InvoiceService and the InvoiceService uses the correct ICurrencyStrategy to convert the invoice amount.

3.Non-Functional Requirements & Scalability

3.1. Availability

Availability means how often the system is up and running (Microsoft, 2026). TechMove Logistics needs the system that will always be available because freight operations happen 24/7 across different time zones. If the system goes down the contracts can be missed and data can be lost. The system targets 99.9% availability, meaning that it can only be down for a few hours per year. To reach this, the system runs across two cloud zones. If one zone fails, the other takes over automatically without manual help needed.

Health checks run every 60 seconds. If anything goes offline the IT team will get an alert straight away. The database will use SQL Server AlwaysOn which keeps a backup copy ready. If the main database fails, the backup will take over in seconds (Microsoft, 2026). If

the currency API goes down, the system will use the last saved exchange rate instead of crashing and mark those invoices for someone to check later.

3.2. Interoperability

Interoperability refers to the system's ability to communicate and work with other systems both internally at TechMove and with external services (Amazon Web Services, 2025). This is important because the GLMS must connect to a live currency exchange API to convert the invoice amounts and in future it may need to integrate with partner freight platforms or third-party billing tools.

All core GLMS functions such as contracts, service requests and invoices will be exposed through RESTful API using JSON as a standard data format, allowing any external system regardless of platform or programming language to communicate with the GLMS easily. All dates will follow format known as ISO 8601 to eliminate international date confusion (Oracle, 2026). Security is managed through OAuth 2.0, issuing access tokens instead of sharing passwords, keeping the data protected.

Because the currency module uses the strategy design pattern, new conversion methods can be added in future without changing any existing code making the system flexible.

3.3. Scalability strategy

If Techmove suddenly increase and adds 50000 new contracts overnight, the system must handle the sudden increase of users, service requests and invoices without slowing down. The GLMS manages this by using these three automatic strategies.

Strategy 1: Horizontal scaling

Akamai (2026) explains horizontal scaling as adding more application instances instead of upgrading one server. This is different from vertical scaling which needs the server to be taken offline for hardware upgrades, causing downtime that TechMove doesn't want. The GLMS adds more Docker Containers using Azure Kubernetes Service when CPU usage goes above 70% for two minutes. New instances start in less than 30 seconds because the Docker images are already built and stored in azure container registry. When the workload decreases, the extra instances will be removed automatically to save costs.

Strategy 2: Load Balancing

A load balancer sits in front of all application instances. When the user makes a request for example, opening a contract, the load balancer decides which application instance is least busy and sends the request there. No single server gets overwhelmed. If one server crashes, then the load balancer automatically stop sending traffic to it and Azure Application Gateway would be used for this purpose (Paliwal, 2024).

Strategy 3: Caching

Microsoft (2026) describes Redis Cache as an open-source, in-memory data store that delivers sub-millisecond response times, making it one of the effective tools for reducing database load

under peak conditions. The GLMS caches exchange rates with a five-minute Time to live (TTL), active client lists with a fifteen-minute TTL and a dashboard contract summary with two-minute TTL. During 50,000 contract surge, all concurrent invoice conversion read Redis at about two milliseconds per call instead of calling the live API, reducing API calls during migration window. The SQL Server AlwaysOn read replica handles all reporting and dashboard queries, leaving the main database free to handle only write operations during peak load.

The combination of horizontal scaling, load balancing and caching means that adding 50,000 contracts overnight does not cause slow performance. The system simply allocates more resources automatically and serves cached data where possible. Once the migration period is over and load normalises, the extra instances are automatically shut down.

Reference List:

Akamai.,2026.*Horizontal vs.Vertical Scaling.Which is best for APIs?*,[Online].Available at:<https://www.akamai.com/glossary/what-is-horizontal-scaling-vs-vertical-scaling> [Accessed 12 April 2026].

Amazon Web Services.,2025.*What is Interoperability*.[Online]Available at: <https://aws.amazon.com/what-is/interoperability/> [Accessed 11 April 2026].

Edmead.M.,2025.*The Zachman Framework Explained*.[Online].Available at: <https://www.linkedin.com/pulse/zachman-framework-explained-mark-edmead-5xihc> [Accessed 10 April,2026]

Geeksforgeeks.,2026.*Factory Method Pattern*,[Online]Available at: <https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/> [Accessed 12 April 2026].

Microsoft.,2026.What is Azure Cache for Redis?,[Online].Available at:<https://learn.microsoft.com/en-us/azure/azure-cache-for-redis/cache-overview> [Accessed 10 April 2026].

Microsoft.,2026.What are availability zones[Online]Available at: <https://learn.microsoft.com/en-us/azure/reliability/availability-zones-overview?tabs=azure-cli> [Accessed 11 April 2026].

Subbareddy,O.,2025.Understanding Gang of Four Design Patterns,[Online] 21 March.Available at:<https://medium.com/@EngineeringVision/understanding-gang-of-four-gof-design-patterns-efe4ef0bf18e> [Accessed 11 April 2026].

Oracle.,2026.*ISO 8601 Date,Time, and duration Support*.[Online].Available at:<https://docs.oracle.com/en/database/oracle/oracle-database/21/adjsn/iso-8601-date-time-and-duration-support.html> [Accessed 12 April 2026].

Paliwal.Y.,2024.Scalability and Load Balancing:The backbone of modern System Design,[Online]Available at: <https://medium.com/@yashpaliwal42/scalability-and-load-balancing-the-backbone-of-modern-system-design-8444619f8745> [Accessed 10 April 2026].